Request for Comments

UME-RFC-001

UME Organization Operating System

Architecture, Module Protocol & Extension Model

STATUS: PROPOSED — OPEN FOR COMMENT

Comment Period: 60 days from March 2026 | Submit comments to: rfc@ume.io

RFC Number	UME-RFC-001
Title	UME Organization Operating System — Architecture, Module Protocol, and Extension Model
Status	PROPOSED — Open for Comment
Category	Standards Track — Platform Architecture
Authors	UME Platform Team
Created	March 2026
Last Updated	March 2026
Comment Period	60 days from publication
Supersedes	None — founding RFC
Related Documents	UME-DA-001, UME-DA-002, UME-LLD-001, UME-PRD-001, UME-PRD-PROB-001

Abstract

This RFC defines the UME Organization Operating System: a Rust-based platform providing a unified kernel and 42 organization modules covering every operational domain of any organization. This document specifies the kernel architecture, the module protocol (the KernelModule trait and its behavioral requirements), the event bus contract, the RBAC model, the storage abstraction, the audit chain protocol, and the extension model for custom modules.

This RFC is a Standards Track document. If accepted, all platform components, all built-in modules, and all custom modules distributed through the UME marketplace must comply with the specifications defined herein.

Portions of this RFC are marked with [OPEN QUESTION] where the authors have identified design decisions that benefit from broader community input before being finalized.

Copyright Notice

Copyright (C) 2026 UME Platform Team. All rights reserved. This document may be reproduced and distributed in whole or in part, without restriction of any kind, provided that the above copyright notice and this paragraph are included in all such copies.

Table of Contents

1 Introduction
1.1 Motivation
1.2 Scope
1.3 Terminology
1.4 Document Conventions
2 System Architecture
2.1 Architecture Principles
2.2 Component Model
2.3 Crate Dependency Rules
2.4 Process Model
3 Kernel Specification
3.1 Kernel Lifecycle
3.2 Boot Protocol
3.3 Configuration Contract
4 Module Protocol (KernelModule)
4.1 Trait Specification
4.2 ModuleDescriptor
4.3 Lifecycle State Machine
4.4 Behavioral Requirements
4.5 Event Naming Convention
5 RBAC Specification
5.1 Permission Model
5.2 Role Hierarchy
5.3 Resource Scoping
5.4 Enforcement Points
6 Event Bus Contract
6.1 Topic Naming
6.2 EventEnvelope
6.3 Delivery Guarantees
6.4 Dead-Letter Queue
6.5 Idempotency
7 Storage Abstraction
7.1 StorageManager Trait
7.2 Transaction Semantics
7.3 Multi-Tenancy Invariant
7.4 Schema Migration Protocol
8 Audit Chain Protocol
8.1 Record Structure
8.2 Chain Integrity
8.3 Mandatory Audit Events
9 Device Driver Specification
9.1 DeviceDriver Trait
9.2 DeviceClass Registry
9.3 Health & Circuit Breaker
10 Extension Model
10.1 Custom Module Namespacing
10.2 SDK Contract
10.3 Module Signing
10.4 Marketplace Metadata
11 HTTP API Contract

11.1 Base Conventions
11.2 Authentication Protocol
11.3 Response Envelopes
11.4 Error Format
12 Backward Compatibility Policy
12.1 Stability Tiers
12.2 Deprecation Protocol
12.3 Versioning
13 Open Questions
14 Security Considerations
15 References

1. Introduction

1.1 Motivation

Organizations of all types — corporations, non-profits, governments, cooperatives, investment funds — share a common set of operational needs: managing their structure and legal obligations, tracking their finances, governing their people, managing risk, and executing their strategy. Yet no platform has ever provided a unified foundation for all of these needs simultaneously.

The result is organizational fragmentation: every organization runs on a patchwork of 10–30 disconnected tools, none of which shares a common model of the organization itself. This fragmentation is the root cause of missed compliance deadlines, financial opacity, manual reconciliation work, and the inability to detect cross-domain risks and opportunities.

UME is designed to solve this problem by providing a kernel-mediated organization operating system. The kernel provides shared infrastructure — identity, storage, events, RBAC, audit — and 42 built-in organization modules provide domain-specific capabilities. All modules share a single organizational model, communicate via a common event bus, and are governed by a single RBAC engine.

1.2 Scope

This RFC specifies:

- The kernel architecture and its mandatory subsystems
- The KernelModule trait: the contract every module must satisfy
- The RBAC model: permission structure, role hierarchy, resource scoping
- The event bus contract: topic naming, envelope structure, delivery guarantees
- The storage abstraction: StorageManager trait, transaction semantics, multi-tenancy invariant
- The audit chain protocol: record structure, chain integrity, mandatory events
- The device driver specification: DeviceDriver trait and DeviceClass registry
- The extension model: custom module namespacing, SDK contract, signing, marketplace
- The HTTP API contract: conventions, authentication, response envelopes
- The backward compatibility policy: stability tiers, deprecation, versioning

This RFC does not specify the implementation of any individual organization module. Module-level design is specified in UME-LLD-001.

1.3 Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 (Bradner, 1997).

Term	Definition
Kernel	The UME runtime process: composition root, all subsystems, and the module registry
Module	A Rust crate implementing KernelModule, registered with the kernel, providing a domain capability

ume_core	The shared Rust crate defining all cross-module types, traits, and contracts
Organization	The top-level entity that owns a UME deployment; corresponds to a business or institution
Subject	An entity that can be authorized: a User, ServiceAccount, or ApiKey
Permission	A named, versioned capability declaration in the format {domain}.{resource}.{verb}
Domain Event	An immutable fact published to the event bus after a state change, in format {domain}.{resource}.{verb}
Policy Pack	A versioned set of compliance rules for a specific jurisdiction and entity type, loaded by Chombo
DLQ	Dead-Letter Queue: destination for events or messages that failed delivery after all retries
Facade	A narrow interface provided to modules by the kernel; modules MUST NOT bypass the facade

1.4 Document Conventions

Rust type names and trait identifiers are written in CamelCase. Module identifiers use snake_case. Event topic strings use the {domain}.{resource}.{verb} format. SQL identifiers use lowercase with underscores. Code examples are illustrative; they represent the intended semantics and MUST be followed, but minor syntactic variations are acceptable.

2. System Architecture

2.1 Architecture Principles

All platform components MUST be designed in accordance with these principles, listed in priority order. Where a design decision requires trading off between principles, higher-priority principles take precedence.

Priority	Principle	Binding Rule
1	Kernel Mediation	Modules MUST NOT access any external resource (storage, network, identity) except through registered device drivers via the kernel facade.
2	Domain Isolation	Domain crates (organization modules) MUST NOT depend on ume_kernel. They MUST depend only on ume_core.
3	RBAC Supremacy	Authorization decisions MUST be made by the kernel RBAC engine. Modules MUST NOT implement alternative authorization logic.
4	Audit Completeness	Every state-changing operation MUST produce an audit record. Audit records MUST be immutable after write.
5	Fault Isolation	A module in Error or Stopped state MUST NOT prevent other modules or the kernel from operating.
6	Event-Driven Integration	Modules MUST communicate across domain boundaries via the event bus. Direct module-to-module function calls are PROHIBITED.
7	Schema Primacy	All cross-module domain types MUST be defined in ume_core before any module implements them.
8	Composability	Any subset of modules MUST be activatable without affecting the correctness of active modules.

2.2 Component Model

Layer Architecture (bottom to top)

Layer 0 — ume_core Shared types, traits, errors, events, schemas. No external dependencies.
Layer 1 — ume_kernel Kernel implementation: DeviceBus, RBAC, EventBus, Executors, Storage, Audit, Supervisor.
Depends on: ume_core
Layer 2 — ume_modules 42 organization modules. Each is an independent crate.
Depends on: ume_core ONLY. MUST NOT depend on ume_kernel.
Layer 3 — ume_server HTTP API server (Axum). Depends on: ume_kernel, ume_modules, ume_core.
Layer 3 — ume_runtime CLI runtime. Depends on: ume_kernel, ume_modules, ume_core.
Layer 3 — ume_sdk Custom module SDK. Depends on: ume_core ONLY.
Layer 4 — clients/java Java client suite. Communicates via ume_server HTTP/WS only.
Layer 4 — custom modules Third-party modules. Depend on: ume_core (via ume_sdk) ONLY.

2.3 Crate Dependency Rules

The following dependency rules are MANDATORY. Violations MUST be detected and rejected in CI:

1. ume_core MUST have zero workspace-internal dependencies.
2. ume_kernel MUST depend only on ume_core from the workspace.
3. Every module crate in ume_modules MUST depend on ume_core and MUST NOT depend on ume_kernel.
4. ume_sdk MUST depend only on ume_core from the workspace.
5. Custom modules MUST declare a dependency on ume_sdk. They MUST NOT depend on ume_kernel directly.

These rules are enforced using cargo-deny or equivalent tooling. A PR that introduces a forbidden dependency MUST be rejected by CI before review.

2.4 Process Model

UME runs as a single operating system process per deployment node. There is no mandatory microservice split. All modules share the same memory space, communicate via in-process channels (or an external broker when configured), and are managed by the single kernel instance.

For horizontal scaling, multiple UME server instances may be deployed behind a load balancer. In this configuration, the event bus MUST be configured to use an external broker (NATS, Kafka, or RabbitMQ) so that events are visible to all instances. The storage tier is shared (PostgreSQL). Session state MUST NOT be stored in-process.

3. Kernel Specification

3.1 Kernel Lifecycle

The kernel MUST implement exactly the following lifecycle states, with the following permitted transitions:

State	From	Trigger	Side Effects
Booting	—	Process starts; UmeKernel::boot() called	Initialize subsystems in dependency order
Running	Booting	All subsystems and required modules reach Running	Emit kernel.ready event; begin accepting API requests
Degraded	Running	One or more modules in Error/Stopped state; kernel still serving	Emit kernel.degraded event; alert operators; continue operating
ShuttingDown	Running | Degraded	SIGTERM or graceful shutdown command received	Drain in-flight operations; stop modules in dependency-reverse order
Stopped	ShuttingDown	All modules stopped; all resources released	Process exits with code 0

3.2 Boot Protocol

The kernel MUST execute the boot sequence in the following order. Any step failure at steps 1–9 MUST abort the boot with a structured error message. Steps 10–14 MAY produce warnings without aborting boot.

Step	Subsystem	Failure Mode
1. Validate configuration	KernelConfig	ABORT — invalid config must not proceed
2. Initialize MemoryManager	MemoryManager	ABORT — memory regions are required for all subsequent steps
3. Initialize LogAuditManager	LogAuditManager	ABORT — audit is non-optional; system cannot run without it
4. Initialize DeviceBus, register drivers	DeviceBus	WARN per driver failure; ABORT if storage driver fails
5. Initialize StorageManager	StorageManager	ABORT — storage is non-optional
6. Run schema migrations	StorageManager.run_migrations()	ABORT — running on stale schema is unsafe
7. Initialize RbacEngine	RbacEngine	ABORT — authorization is non-optional
8. Initialize EventBus	EventBus	ABORT — event bus is non-optional
9. Initialize ExecutorPool	ExecutorPool	ABORT — executors are non-optional
10. Initialize JobScheduler	JobScheduler	ABORT — scheduler is non-optional
11. Build ModuleRegistry; resolve dependency graph	ModuleRegistry	ABORT if dependency cycle detected; otherwise continue
12. Start modules in topological order	Each KernelModule	WARN per non-critical module failure; continue with remaining
13. Start SupervisorEngine	SupervisorEngine	ABORT — supervision is non-optional

14. Emit kernel.ready; accept requests	EventBus, ume_server	Ready — boot complete

3.3 Configuration Contract

All configuration MUST be injectable via environment variables (prefixed UME_). Configuration files (TOML) are supported as a secondary source. The precedence order, highest to lowest, is: environment variables, config file, compiled-in defaults.

Sensitive values (database passwords, API keys, encryption keys) MUST be read from environment variables or mounted secret files only. They MUST NOT appear in config files that could be committed to version control.

4. Module Protocol (KernelModule)

4.1 Trait Specification

Every organization module MUST implement the following trait exactly as specified. The method signatures are normative. Implementations MUST satisfy all behavioral requirements in §4.4.

Method	Return Type	Specification
id()	&'static str	MUST return a globally unique, stable module identifier. Format: "{domain}.{name}". MUST NOT change between versions.
domain()	Domain	MUST return the domain area this module belongs to.
version()	&'static str	MUST return a semantic version string (MAJOR.MINOR.PATCH).
state()	ModuleState	MUST return the current lifecycle state. MUST be accurate within one supervisor interval.
start(kernel)	Result<(),UmeError>	MUST complete initialization. MUST NOT block the calling thread for more than the boot_timeout. MUST return Err if a required resource is unavailable.
stop()	Result<(),UmeError>	MUST complete within drain_timeout (default 30s). MUST release all subscriptions, cancel scheduled jobs, and complete or abort in-flight operations.
health_check()	HealthStatus	MUST return within 500ms. A timeout is recorded as a health check failure. MUST return Unhealthy if a required driver is unavailable.
describe()	ModuleDescriptor	MUST return a complete, accurate descriptor. Descriptor MUST be stable; it MUST NOT change after start() has been called.
on_event(envelope)	Result<(),UmeError>	MUST handle events for all topics declared in ModuleDescriptor.subscriptions. MUST NOT block the event executor for more than 5 seconds.
dependencies()	Vec<&'static str>	MUST list all module IDs that must be in Running state before this module can start.
configure(config)	Result<bool,UmeError>	MUST apply configuration changes without restart where possible. MUST return Ok(true) if a restart is required, Ok(false) if hot-reload succeeded.

4.2 ModuleDescriptor

The ModuleDescriptor is a module's self-declaration. It is the primary input for the kernel's dependency resolution, RBAC registration, and event bus subscription setup.

Field	Type	Required	Description
id	&'static str	Yes	Same value as id(). MUST be globally unique.
domain	Domain	Yes	Domain enum value matching the module's primary area.
version	&'static str	Yes	Semantic version string.
description	&'static str	Yes	Human-readable description for the marketplace and health dashboard.

dependencies	Vec<&'static str>	Yes	Module IDs that MUST be Running before this module can start.
permissions	Vec<Permission>	Yes	All permissions this module registers in the RBAC system. Empty if the module performs no authorization.
subscriptions	Vec<TopicPattern>	Yes	Event bus topic patterns this module subscribes to. Empty if the module is not event-driven.
config_schema	Vec<ConfigDescriptor>	Yes	Schema for all configurable values. Used to validate configuration and generate UI.
capabilities	Vec<&'static str>	Yes	Discoverable capability tags for marketplace and inter-module discovery.

4.3 Lifecycle State Machine

State	From	Trigger	Side Effects
Registered	(initial)	Module registered in registry	Descriptor loaded; dependencies recorded; permissions registered in RBAC
Initializing	Registered	Dependencies being resolved	Kernel waits for all declared dependency modules to reach Running
Starting	Initializing	All dependencies Running	Kernel calls start(kernel_facade)
Running	Starting	start() returns Ok(())	Module begins processing events and serving requests
Degraded	Running	health_check() returns Warning	Supervisor logs warning; continues to call health_check; module keeps serving
Error	Running | Degraded	health_check() returns Unhealthy, or timeout	Supervisor applies restart policy; module MUST NOT serve new requests
Stopping	Running | Degraded | Error	stop() called	Drain timeout starts; module completing/aborting in-flight work
Stopped	Stopping	stop() returns, or drain_timeout exceeded	Resources released; module can be restarted by supervisor

4.4 Behavioral Requirements

These behavioral requirements apply to ALL module implementations without exception:

6. Every module MUST call kernel.assert_can() before executing any mutating operation.
7. Every module MUST call kernel.audit() after every successful create, update, delete, or approval operation.
8. Every module MUST publish a domain event via kernel.publish() after every successful state-changing operation.
9. Modules MUST NOT hold references to kernel subsystems beyond the scope of a single operation. The kernel facade MUST be re-acquired on each call.
10. Modules MUST NOT spawn threads or async tasks outside of the executor pool. All background work MUST be submitted to the appropriate named executor.
11. Modules MUST NOT access the filesystem directly. All storage MUST go through the StorageManager facade.

12. Modules MUST NOT make outbound network calls directly. All network I/O MUST go through registered device drivers.
13. Modules MUST implement idempotent on_event() handlers. The event bus MAY deliver the same event more than once.
14. Modules MUST NOT produce events on the kernel.* topic namespace. That namespace is reserved for kernel-internal events.

4.5 Event Naming Convention

All domain events published by modules MUST follow this naming convention:

```
{domain}.{resource}.{verb}
  domain   — the module's domain area, lowercase (e.g., finance, hr, risk, chombo)
  resource — the entity or aggregate being acted upon, lowercase (e.g., journal, employee, entity)
  verb     — past tense, describing what happened (e.g., posted, hired, evaluated, overdue)

  Examples:
    finance.journal.posted       finance.invoice.approved       finance.period.locked
    hr.employee.hired            hr.employee.terminated         hr.payroll.completed
    risk.kri.red_threshold       risk.risk.appetite_breached
    chombo.entity.evaluated      chombo.filing.due              chombo.filing.overdue
    security.incident.created    security.identity.login

  Invalid examples:
    finance.postJournal  (not past tense verb)    user.created  (not scoped to domain)
    FINANCE.journal.posted  (uppercase)           journal.posted  (missing domain)
```

5. RBAC Specification

5.1 Permission Model

Permissions are named capability declarations in the format {domain}.{resource}.{verb}. Every permission MUST be declared in the ModuleDescriptor of the module that owns the capability before it can be granted to a role.

Element	Rules
Permission ID format	{domain}.{resource}.{verb} — all lowercase, dot-separated, max 100 characters
Domain segment	MUST match the declaring module's domain area
Wildcard	A permission ending in .* grants all permissions in that namespace to a role (e.g., finance.* = all finance permissions)
resource_scoped flag	If true, the permission may be granted scoped to a specific resource ID, in addition to a global grant
Admin permission	Each module SHOULD declare a {domain}.admin permission granting full access to that module

5.2 Role Hierarchy

Roles MAY declare parent roles. A subject with a role inherits all permissions of all ancestor roles. The hierarchy MUST be a DAG (directed acyclic graph). A cycle in the role hierarchy MUST cause UmeError::RbacCycleDetected and MUST prevent the offending role configuration from being applied.

Built-in platform roles (prefix: ume.):

Role ID	Description	Default Inheritance
ume.super_admin	Full platform administration: all permissions, including kernel.admin	None
ume.org_admin	Organization-wide administration: all org module permissions	ume.user
ume.finance_admin	Full access to all Finance module capabilities	ume.viewer
ume.hr_admin	Full access to all HR module capabilities	ume.viewer
ume.compliance_officer	Full access to GRC, Risk, and Chombo modules	ume.viewer
ume.read_only	Read-only access to all modules (all *.view permissions)	None
ume.user	Standard employee: portal, work, learning, HR self-service	ume.read_only
ume.viewer	Dashboard and report viewing only	None

5.3 Resource Scoping

Permissions marked resource_scoped: true may be granted at two granularities:

- Global grant: subject can perform the action on all instances of the resource type
- Resource-scoped grant: subject can perform the action only on the specific resource ID named in the grant

Example: finance.budget.manage granted globally allows managing all budget lines. Granted with resource_id = "cost-center-uuid-123" allows managing only that cost center's budget lines. Both may be granted simultaneously for different resource sets.

5.4 Enforcement Points

The following enforcement rules are MANDATORY:

15. The kernel RBAC engine is the ONLY valid authorization decision point. Modules MUST NOT perform their own authorization checks.
16. kernel.assert_can() MUST be called before every mutating operation, without exception.
17. A denied authorization request MUST produce an audit record regardless of module-level audit configuration.
18. The result of assert_can() MUST NOT be cached by the calling module. Caching is handled by the kernel RBAC engine internally.
19. Service-to-service calls (module A's event handler triggering module B's operation) MUST use a ServiceAccount subject with a declared set of permissions, not a elevated bypass.

6. Event Bus Contract

6.1 Topic Naming

All event topics MUST conform to the format {domain}.{resource}.{verb} as specified in §4.5. The kernel event bus MUST reject publications to topics that do not conform to this format with UmeError::ValidationFailed.

The following topic namespaces are reserved and MUST NOT be published to by modules:

- kernel.* — reserved for kernel lifecycle events
- audit.* — reserved for audit record notifications
- dlq.* — reserved for dead-letter queue routing

6.2 EventEnvelope

Every event published to the bus MUST be wrapped in an EventEnvelope with the following fields:

Field	Type	Required	Description
id	EventId (UUID v4)	Yes	Unique event identifier. Assigned by the kernel on publish.
topic	String	Yes	Must match {domain}.{resource}.{verb} format.
payload	serde_json::Value	Yes	Event-specific data. Schema is module-defined and documented in the module's changelog.
published_at	DateTime<Utc>	Yes	UTC timestamp assigned by the kernel at publish time. Modules MUST NOT override this.
publisher_module	String	Yes	Module ID of the publisher. Assigned by the kernel from the calling module's identity.
correlation_id	Option<String>	No	Request or operation ID that caused this event. Used for distributed tracing.
causation_id	Option<EventId>	No	ID of the event that triggered this event (for event chains).
idempotency_key	Option<String>	No	If set, the bus MUST deduplicate events with the same key within the dedup window (default 5 minutes).

6.3 Delivery Guarantees

Backend	Delivery Guarantee	Ordering Guarantee	Notes
InMemory (default)	At-most-once (no persistence)	FIFO per topic	Suitable for single-node; events lost on restart
NATS JetStream	At-least-once	FIFO per subject	Recommended for production single-region
Kafka	At-least-once	Ordered per partition key	Recommended for high-throughput or multi-region
RabbitMQ	At-least-once	Per-queue FIFO	Suitable for existing RabbitMQ deployments

All event handlers MUST be implemented as idempotent operations. The platform does not guarantee exactly-once delivery on any backend.

6.4 Dead-Letter Queue

Events that fail delivery after all configured retries (default: 5 attempts with exponential backoff starting at 1 second) MUST be routed to the DLQ topic: dlq.{original_topic}. DLQ entries MUST be retained for the configured DLQ retention period (default 30 days) and MUST be inspectable and replayable via the ume_runtime CLI.

6.5 Idempotency

When an EventEnvelope carries an idempotency_key, the bus MUST NOT deliver a second event with the same key to any subscriber within the dedup window. The dedup window MUST be configurable and MUST default to 5 minutes. The bus MUST acknowledge the duplicate publish as successful to the caller (publishers MUST NOT be penalized for duplicates).

7. Storage Abstraction

7.1 StorageManager Trait

All persistence operations MUST go through the StorageManager facade. Modules MUST NOT hold direct database connections, execute raw SQL, or access the filesystem for data storage. The StorageManager is acquired via kernel.storage().await.

Operation	Description	Required
get(namespace, id)	Retrieve a single value by namespace and ID. Returns Option<T>.	Yes
put(namespace, id, value, version)	Upsert a value. If version is Some, MUST perform optimistic lock check; return UmeError::OptimisticLockConflict if version mismatch.	Yes
delete(namespace, id)	Soft-delete a value (set deleted_at). Hard-delete available only via audit-logged admin operation.	Yes
query(namespace, filter, page)	List values matching filter with cursor pagination. Filter is type-safe (Filter enum); raw SQL strings are NOT accepted.	Yes
transaction(f)	Execute a closure within a database transaction. ACID semantics on supported backends.	Yes
run_migrations()	Apply all pending migrations in order. Idempotent. Called by kernel during boot.	Yes

7.2 Transaction Semantics

Transactions MUST use serializable isolation on PostgreSQL. On SQLite, serializable isolation is default. On InMemory, operations are serialized by the store's internal mutex. Operations that span multiple namespaces within a transaction MUST be atomic — if any operation fails, all preceding operations in the transaction MUST be rolled back.

7.3 Multi-Tenancy Invariant

This is a HARD INVARIANT: every query executed by the StorageManager MUST be scoped to the org_id of the current kernel instance. The org_id filter MUST be injected by the StorageManager unconditionally. Modules MUST NOT pass org_id as part of their filter — it is added automatically. A query that returns records from a different org_id MUST be treated as a security-critical bug.

7.4 Schema Migration Protocol

Migrations MUST be:

20. Versioned with a unique identifier: {module}_{sequence:04}_{description} (e.g., finance_0001_initial_schema)
21. Idempotent: running a migration twice MUST produce the same result as running it once
22. Forward-only: no rollback scripts are maintained; issues discovered after migration are fixed by new forward migrations
23. Recorded: every applied migration MUST be recorded in migrations.migration_log with its checksum, timestamp, and duration

24. Verified: the checksum of each migration file MUST be verified against the recorded checksum on every boot; a mismatch MUST cause boot abort

8. Audit Chain Protocol

8.1 Record Structure

Every audit record MUST contain the following fields. Records are immutable after write — no UPDATE or DELETE path exists to the audit_records table.

Field	Type	Required	Description
id	UUID v4	Yes	Unique record identifier.
at	DateTime<Utc>	Yes	UTC timestamp of the action, assigned by the kernel.
actor	SubjectId	Yes	Identity of the subject that performed the action.
actor_type	SubjectType	Yes	User | ServiceAccount | Scheduler | ApiKey.
action	String	Yes	Format: {domain}.{resource}.{verb}. MUST match the event topic of the triggering operation.
resource_id	Option<String>	No	UUID of the primary resource affected. Required for operations on specific resources.
resource_type	String	Yes	Human-readable resource type, e.g., "finance.JournalEntry".
outcome	AuditOutcome	Yes	Success | Failure | Denied.
detail	JSON	Yes	Action-specific detail. MUST NOT contain PII in plaintext. MUST NOT contain credentials.
session_id	Option<String>	No	HTTP session or CLI session identifier.
request_id	Option<String>	No	HTTP request correlation ID.
org_id	OrgId	Yes	Always set. Enables multi-tenant audit isolation.
previous_hash	Option<String>	Yes (after first record)	SHA-256 hash of the previous audit record in the chain.
self_hash	String	Yes	SHA-256 hash of this record, computed after setting previous_hash.

8.2 Chain Integrity

Audit records MUST be cryptographically chained. The chain enables detection of any tampering:

25. The self_hash field MUST be the SHA-256 hash of the record serialized to canonical JSON with self_hash excluded from the input
26. The previous_hash field of record N MUST equal the self_hash of record N-1
27. The first record in the audit log has previous_hash = null
28. The kernel MUST provide a verify_chain(start_id, end_id) operation that checks every link in the specified range
29. verify_chain MUST be executable as an offline verification tool against an audit log export

8.3 Mandatory Audit Events

The following events MUST always produce audit records, regardless of any module-level audit configuration:

Event Class	Required Fields	Notes
RBAC authorization denied	actor, action, resource_id, outcome=Denied	MUST be logged even if the calling module has audit disabled
Authentication event (success, failure, lockout)	actor, action, ip_address, mfa_used	MUST include IP address and MFA status
RBAC permission grant/revoke	actor, subject_affected, permission, resource_id	Sensitive — requires 4-eyes enforcement for privileged permissions
Kernel configuration change	actor, changed_key, old_value_hash, new_value_hash	Values are hashed; plaintext config values MUST NOT appear in audit
Module start/stop	actor (Scheduler), module_id, reason	Enables detection of unauthorized module deactivation
Cryptographic key operation (rotate, use)	actor, key_id, key_version, operation	Key operations are always audit-mandatory

9. Device Driver Specification

9.1 DeviceDriver Trait

All device drivers MUST implement the DeviceDriver base trait. Specialized driver types (StorageDriver, IdentityDriver, CommDriver, etc.) extend this base trait with domain-specific operations.

Method	Specification
device_id()	MUST return a unique, stable string identifier. Format: "{class}.{implementation}", e.g., "storage.postgres".
device_class()	MUST return the DeviceClass enum value for this driver.
status()	MUST return the current DriverStatus: Unknown | Initializing | Healthy | Degraded | Unavailable.
initialize(config)	MUST complete initialization within the configured timeout (default 30s). MUST validate all required config fields.
shutdown()	MUST release all connections and resources. MUST NOT block for more than the configured drain_timeout.
health_check()	MUST return DriverHealth within 5 seconds. MUST test actual connectivity, not just local state.

9.2 DeviceClass Registry

DeviceClass	Purpose	Required Minimum
Storage	Persistence backend (SQL, KV, object)	One instance of Storage MUST be registered at boot
Identity	Authentication and identity resolution (LDAP, OIDC, SAML, local)	One instance SHOULD be registered; LocalIdentityDriver is built-in
Comms	Message dispatch (email, SMS, push, chat)	Optional; absence degrades notification capability
Broker	External event broker (NATS, Kafka, RabbitMQ)	Optional; absence defaults to InMemory bus
Crypto	Cryptographic vault and key management	One instance MUST be registered at boot (default: SoftwareVault)
Search	Full-text search index (PostgreSQL tsvector, Elasticsearch)	Optional; absence degrades search to basic filter queries
Finance	ERP and banking integrations	Optional; enables bank reconciliation and ERP sync
ObjectStorage	Binary/file storage (S3, GCS, Azure Blob)	Optional; required for document management at scale

9.3 Health and Circuit Breaker

The kernel MUST implement circuit breaker semantics for every device driver:

Circuit State	Trigger	Behaviour	Exit Condition

Closed (normal)	Initial state	All calls pass through to the driver	—
Open	N consecutive health_check failures (default: 3)	All calls immediately return UmeError::DriverUnavailable without calling driver	Cooldown period expires (default: 60s)
HalfOpen	Cooldown expires from Open	One probe call is allowed through	Success → Closed; Failure → resets to Open with new cooldown

10. Extension Model

10.1 Custom Module Namespacing

All module IDs MUST use one of the following namespace prefixes:

Namespace Prefix	Usage	Who May Use It
(none — bare domain.name)	Built-in UME platform modules only	UME Platform Team
custom.ume.{vendor}.{name}	UME ecosystem modules distributed via the marketplace	Third-party developers; verified via signing
org.custom.{vendor}.{name}	Organization-private modules; not distributed	The deploying organization

The {vendor} segment MUST be a registered vendor identifier. Vendor registration is required before marketplace distribution. Unregistered vendor identifiers MAY be used for org.custom.* modules.

10.2 SDK Contract

The UME SDK (ume_sdk crate) MUST provide:

30. Complete Rust trait definitions for all kernel facades (KernelFacade, StorageManager, RbacEngine, EventBus, LogAuditManager, all driver types)
31. Mock implementations of all kernel facades for use in module unit and integration tests
32. A MockKernel struct combining all mocks, usable without a running kernel process
33. EventCapture: a test utility that records all events published during a test for assertion
34. A scaffold generator (tools/generate_custom_module) producing a complete, compilable module skeleton
35. Schema validator: a compile-time check that module data models comply with ume_core conventions

The SDK MUST be versioned independently from ume_kernel. A module compiled against SDK version N MUST be binary-compatible with kernel versions N and N+1 (forward-compatible for one minor version).

10.3 Module Signing

Modules distributed via the marketplace MUST be signed with the vendor's registered signing key. The kernel MUST verify the signature before loading a signed module. The kernel MAY be configured to:

- Reject all unsigned modules (recommended for production)
- Log a warning and load unsigned modules (suitable for development)
- Reject modules from unregistered vendors

Modules signed by a revoked key MUST be rejected. The kernel MUST check signature validity against the key revocation list on each module load.

10.4 Marketplace Metadata

Modules distributed via the UME marketplace MUST declare the following metadata in their Cargo.toml metadata section:

Metadata Key	Required	Description
ume.module_id	Yes	Full namespaced module ID
ume.display_name	Yes	Human-readable module name for the marketplace UI
ume.vendor	Yes	Registered vendor name
ume.description	Yes	1–2 sentence description (max 300 chars)
ume.category	Yes	Primary category: Finance | HR | Legal | GRC | Sales | Operations | Technology | Sector
ume.capabilities	Yes	Array of capability tags
ume.kernel_compatibility	Yes	Semver range of compatible kernel versions, e.g., "^1.0"
ume.permissions_requested	Yes	Array of permission IDs this module registers
ume.support_url	Yes	URL for documentation and support
ume.license	Yes	SPDX license identifier

11. HTTP API Contract

11.1 Base Conventions

Convention	Specification
Base path	All module endpoints are prefixed /v1/{domain}. The v1 prefix is the API version, not the module version.
HTTP verbs	GET for reads (MUST be idempotent), POST for create, PUT/PATCH for update, DELETE for soft-delete.
Content type	All requests and responses MUST use Content-Type: application/json.
Datetime format	All datetime fields MUST use ISO 8601 UTC format (RFC3339), e.g., 2026-03-07T14:30:00Z.
IDs	All resource IDs in requests and responses MUST be UUID v4 strings.
Pagination	All list endpoints MUST use cursor-based pagination. Offset pagination is PROHIBITED.
Filtering	All list endpoints SHOULD support filtering via the filter query parameter. Filter syntax is documented per endpoint.

11.2 Authentication Protocol

All non-public endpoints MUST require a valid JWT access token in the Authorization: Bearer {token} header. The token MUST be an RS256-signed JWT issued by the UME Security Module. The following claims MUST be verified on every request:

36. exp: token is not expired
37. iss: issuer matches the expected ume:{org_id} value
38. aud: audience is "ume-api"
39. jti: token has not been revoked (checked against session store)

11.3 Response Envelopes

All API responses MUST use one of the following envelope structures:

Response Type	Structure	When Used
Single resource	{ data: T, meta: ResponseMeta }	GET /resource/{id}, POST (create), PUT (update)
List / paginated	{ data: T[], pagination: PaginationMeta, meta: ResponseMeta }	GET /resource
Empty success	{ meta: ResponseMeta }	DELETE, action endpoints with no return value
Error	{ error_code, message, detail: FieldError[], request_id, documentation_url }	All 4xx and 5xx responses

ResponseMeta MUST include: request_id (UUID), api_version (string), generated_at (RFC3339 UTC).
PaginationMeta MUST include: cursor (next page cursor, null if no more pages), has_more (bool).

11.4 Error Format

All error responses MUST use HTTP status codes per RFC 7231 and MUST include a machine-readable error_code. The following error_code values are standardized:

error_code	HTTP Status	Meaning
UNAUTHENTICATED	401	No valid authentication token provided
FORBIDDEN	403	Valid token, insufficient permissions
NOT_FOUND	404	Resource does not exist
CONFLICT	409	Optimistic lock conflict; resource was modified since last read
VALIDATION_FAILED	422	Request body failed schema or domain validation; detail contains field errors
RATE_LIMITED	429	Request rate limit exceeded; Retry-After header included
PERIOD_LOCKED	422	Attempted write to a locked accounting period
MODULE_UNAVAILABLE	503	Target module is in Error or Stopped state
INTERNAL_ERROR	500	Unexpected internal error; safe to retry after brief delay

12. Backward Compatibility Policy

12.1 Stability Tiers

Tier	Stability Commitment	Examples
Stable	No breaking changes without a major version bump and a minimum 2-minor-version deprecation period	KernelModule trait methods; StorageManager trait; all ume_core public types; HTTP API v1 endpoints
Experimental	May change or be removed with a 1-minor-version notice	New trait methods with default impls; new HTTP endpoints marked as beta
Internal	No stability commitment; not for external use; subject to change at any time	Kernel-internal structs; ume_kernel private APIs; test utilities

12.2 Deprecation Protocol

When a Stable API element is to be changed or removed:

40. A deprecation notice MUST be added to the API element in the release where deprecation begins
41. The deprecated element MUST remain functional for a minimum of 2 minor versions after the deprecation notice
42. A migration guide MUST be published alongside the deprecation notice
43. The CHANGELOG MUST list all deprecated elements in each release
44. Removal MUST NOT occur in a minor version release; it MUST occur in a major version bump

12.3 Versioning

UME uses Semantic Versioning (MAJOR.MINOR.PATCH):

Version Component	When It Increments	Compatibility Guarantee
MAJOR	Breaking changes to any Stable API; major architectural changes	No backward compatibility guaranteed; migration guide required
MINOR	New Stable features added; Experimental features added or changed; deprecations announced	Backward compatible for all Stable APIs; modules compiled on MAJOR.N work on MAJOR.N+1
PATCH	Bug fixes; security fixes; documentation updates	No API changes; drop-in replacement

13. Open Questions

The following design decisions are marked [OPEN QUESTION] in this RFC. The authors invite comments from the community before these are finalized. Comments must be submitted within the 60-day comment period.

OQ ID	Question	Options Considered	RFC Author Preference
OQ-01	Should modules be allowed to communicate via direct facade calls to other modules, in addition to the event bus, for synchronous scenarios?	(A) Events only — strict async; (B) Optional sync facade for registered inter-module contracts	(A) Events only — preserves isolation; synchronous needs can be served by designing compensating events
OQ-02	Should the KernelModule trait support capability negotiation — a module declaring optional capabilities that other modules can query before attempting integration?	(A) No — keep descriptor simple; (B) Yes — add capabilities() method returning a capability map	(B) Yes — reduces integration failures when optional features are absent
OQ-03	Should jurisdiction policy packs be versioned independently of the platform (like npm packages) or bundled in the platform release?	(A) Bundled — simpler; (B) Independent — allows compliance updates without platform upgrade	(B) Independent — compliance obligations change faster than platform releases
OQ-04	Should custom modules be loadable as dynamic libraries (.so files) or as statically linked crates compiled into the binary?	(A) Dynamic — hot-loading at runtime without rebuild; (B) Static — easier, more secure, simpler build	(A) Dynamic — essential for the marketplace model; security addressed by signing
OQ-05	Should the HTTP API expose a GraphQL interface in addition to REST?	(A) REST only; (B) REST + optional GraphQL read layer; (C) GraphQL only	(B) REST primary + GraphQL experimental — REST for integrations; GraphQL for complex client queries
OQ-06	Should the audit chain use SHA-256 (current) or a more collision-resistant algorithm (SHA-3/Blake3)?	(A) SHA-256 — widely supported, current standard; (B) Blake3 — faster, more modern; (C) SHA-3 — NIST standard	(A) SHA-256 — widest regulatory acceptance; move to Blake3 in a future major version

Copyright 2026 UME Platform Team

14. Security Considerations

Implementors of this specification MUST consider the following security properties:

Area	Security Property Required
Authentication	All non-public endpoints require valid JWT. Tokens are RS256-signed. Session revocation is enforced via JTI check on every request.
Authorization	RBAC is enforced by the kernel only. Modules cannot bypass it. RBAC denials are always audited. Least-privilege defaults apply to all new subjects.
Data at rest	All persistent data MUST be encrypted with AES-256-GCM at the storage driver layer. Encryption keys are managed via the CryptoVault interface.
Data in transit	TLS 1.2+ MUST be used for all HTTP API connections in production. HTTP without TLS MUST be disabled in production mode.
Injection prevention	The Filter enum type system prevents SQL injection — raw SQL from module code is architecturally impossible. All API inputs are validated against schema before processing.
Supply chain security	Custom modules MUST be signed. The SBOM for the platform MUST be published with each release. Dependencies MUST be scanned for known CVEs.
Key management	No raw key material may be accessed outside the CryptoVault interface. HSM-compatible by design. Keys are rotatable without downtime.
Audit integrity	Audit records are immutable after write. SHA-256 chain linking enables tamper detection. Chain verification is available as an offline tool.
Multi-tenancy isolation	All queries are unconditionally scoped to org_id by the StorageManager. Cross-organization data access is architecturally impossible in the query layer.

Copyright 2026 UME Platform Team | Open for Comment — 60 days from March 2026

15. References

Reference	Title
RFC 2119	Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997
RFC 7231	Fielding, R., Reschke, J., "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content", RFC 7231, June 2014
RFC 7519	Jones, M., et al., "JSON Web Token (JWT)", RFC 7519, May 2015
RFC 6238	M'Raihi, D., et al., "TOTP: Time-Based One-Time Password Algorithm", RFC 6238, May 2011
SemVer	Preston-Werner, T., "Semantic Versioning 2.0.0", semver.org
UME-DA-001	UME Design & Architecture Document v1.0.0
UME-DA-002	UME Comprehensive Design & Architecture Document v1.0.0
UME-LLD-001	UME Low-Level Software Design Specification v1.0.0
UME-PRD-001	UME Platform Requirements Document v1.0.0
UME-PRD-PROB-001	UME Problem Requirements Document v1.0.0 (this companion document)
UME-WUC-001	UME Users, Workflows & Use Cases v1.0.0
UME-WUC-002	UME Expanded Users, Workflows & Use Cases v1.0.0
OWASP TLS	OWASP, "Transport Layer Security Cheat Sheet", owasp.org
ISO 27001	ISO/IEC 27001:2022, "Information security management systems"
GDPR	Regulation (EU) 2016/679 (General Data Protection Regulation)

End of RFC UME-001 — Awaiting Community Comments

How to Submit Comments

- Email: rfc@ume.io with subject line "Comment: UME-RFC-001 §{section}"
- Include: your name/organization, the section number, the specific text you are commenting on, and your proposed change or question
- Comment period closes 60 days from March 2026
- All substantive comments will be acknowledged and addressed in the final version or deferred to a future revision
- Authors of accepted comments that result in material changes will be acknowledged in the Contributors section